

Atividade Prática 1 — Classificador Bayesiano

Etapa 2 — Massa de Dados de Treinamento

Autor	Marcus Vinícius Santos de Almeida
Disciplina	Mineração de Dados
Data de entrega	04/09/2026
Domínio	Predição de defeitos de software a partir de métricas estáticas de código
Objetivo da etapa	Gerar sinteticamente 100+ registros de treinamento, com padrões intencionais e distribuição de classes razoável — $N = 1.000.000$ nesta versão
Artefatos	<code>dados/gerar_dados.py</code> · <code>dados/etapa2_dados_treinamento.csv</code> · <code>dados/etapa2_validacao.txt</code>

1. Metodologia de Geração

1.1 Por que dados sintéticos

A Etapa 2 do enunciado exige explicitamente que os 100+ registros sejam **gerados por IA / sinteticamente**, e não coletados de repositórios reais. Portanto, usar dados reais aqui descumpriria a atividade. A geração é feita por um **script Python** (`dados/gerar_dados.py`) — não por texto solto produzido por um chat — para que o modelo generativo seja deliberado, inspecionável e reproduzível.

1.2 Decisão de projeto: "Naive Bayes puro" — features independentes entre si

O Naive Bayes assume independência condicional entre as features dado o rótulo. A decisão tomada para este projeto (documentada no `CLAUDE.md`, seção "Naive Bayes puro") foi **gerar as 6 features de forma estatisticamente independente entre si**: nenhuma feature lê o valor de outra no gerador — sem tendência cruzada, sem ruído compartilhado, sem subpopulação de interação plantada. Só o **rótulo** (`defeito = SIM/NÃO`) depende das 6 features, o que é esperado e correto — é literalmente o que o Naive Bayes modela, $P(\text{feature} \mid \text{classe})$.

Por que essa escolha, e não plantar as correlações que a literatura documenta:

- Testa o classificador no cenário em que sua própria suposição central (independência condicional) é **verdadeira por construção** dos dados — o "Naive Bayes em sua forma pura", sem misturar o experimento com o efeito de uma violação plantada.
- É uma escolha de simulação **legítima e declarada como tal** — não uma afirmação sobre como o mundo real funciona. A literatura (Shepperd, 1988; $R^2 \approx 0,93$ entre complexidade ciclomática e LOC) documenta que, em código real, essas features **seriam** correlacionadas. Este dataset sintético opta por não reproduzir essa correlação, de propósito, para isolar o comportamento teórico do algoritmo — a discussão desse contraste (simulação vs. mundo real) é o tema central da Reflexão Crítica da Etapa 4.

Mecanismo: cada uma das 6 features é sorteada por um gerador de números aleatórios **próprio** (`numpy.random.Generator.spawn()`), uma "ramificação" independente da mesma semente `SEED`), evitando inclusive o compartilhamento acidental de ruído entre colunas.

1.3 Parâmetros principais (ajustáveis no topo do script)

Parâmetro	Valor	Significado
<code>N_REGISTROS</code>	1.000.000	Nº de módulos gerados (mínimo exigido: 100). Aumentado da versão anterior (150) — ver <code>CLAUDE.md</code> , "Migração para DuckDB e $N = 1.000.000$ registros"
<code>SEED</code>	42	Semente única — todo gerador de número aleatório do script deriva dela (<code>rng.spawn</code>), garantindo reprodutibilidade
<code>PROPORCAO_DEFEITO</code>	0,35	Fração de módulos com <code>defeito = SIM</code> (longe de 0,5 e dos extremos)

Nota sobre o CSV gerado. Com $N = 1.000.000$, `dados/etapa2_dados_treinamento.csv` passa a ter dezenas de MB e **não é versionado no Git** — está listado no `.gitignore` do projeto. O arquivo é 100 % reproduzível: rodando `python3 dados/gerar_dados.py` localmente, com a mesma semente fixa (`SEED = 42`), qualquer pessoa obtém exatamente o mesmo CSV byte a byte. A avaliação desta atividade é sobre a apresentação oral, não auditoria do repositório Git, então essa decisão não representa risco de entregável ausente (ver `CLAUDE.md`).

1.4 Modelo generativo (resumo)

LOC vem de uma log-normal (muitos arquivos pequenos, poucos grandes). **Complexidade** é uma log-normal própria (mediana $\approx 8,6$), com uma subpopulação aleatória (12%) de complexidade desproporcionalmente alta — um sorteio independente, que não olha LOC nem nenhuma outra coluna. **Nº de autores** é uma contagem de Poisson de média fixa (não depende mais do tamanho). **Churn relativo** é log-normal em torno de $\sim 10,5\%$. **Nº de imports** soma um nível médio fixo $\times$ ruído log-normal forte + uma contagem de Poisson independente. **Cobertura** é normal em torno de 63%, sem depender mais do churn. O **rótulo** é a soma de pesos (log-odds) por categoria das 6 features + ruído gaussiano — sem nenhum bônus de interação entre features; os $k = \text{round}(N \times 0,35)$ módulos de maior score recebem **SIM**, o que mantém a proporção exata mantendo o rótulo estocástico. Cada feature é salva no CSV em **duas formas**: valor bruto e categoria (`baixo/medio/alto`), nas mesmas faixas do `CLAUDE.md` e da Etapa 1.

2. Resultados da Validação

Reproduzível com `python3 dados/gerar_dados.py`; a saída completa está em `dados/etapa2_validacao.txt`.

2.1 Proporção de classes

Classe	Registros	Proporção
SIM (com defeito)	350.000	35,0 %
NAO (sem defeito)	650.000	65,0 %

Distribuição desbalanceada de forma **moderada e proposital** ($\sim 1:2$), próxima do `PROPORCAO_DEFEITO = 0,35` e longe de extremos. Equivalente à proporção da versão anterior do dataset (então 34,7 % / 65,3 %) — o mecanismo top-k ($k = \text{round}(N \times 0,35)$) garante essa fração independentemente de como as features são geradas. Com N maior, `round( $N \times 0,35$ )` bate exatamente em 35,0 % em vez de ficar a uma fração de módulo de distância ($150 \times 0,35 = 52,5$, arredondado para $52 \rightarrow 34,7\%$); é só um efeito de arredondamento em N pequeno, não uma mudança de comportamento do mecanismo.

2.2 Matriz de correlação de Pearson (features numéricas)

	Complex.	LOC	Autores	Churn	Imports	Cobert.
Complex.	1,000	-0,002	-0,002	-0,000	-0,001	0,000
LOC	-0,002	1,000	-0,000	-0,001	-0,001	-0,001
Autores	-0,002	-0,000	1,000	0,000	-0,000	0,000
Churn	-0,000	-0,001	0,000	1,000	-0,001	0,001
Imports	-0,001	-0,001	-0,000	-0,001	1,000	-0,001
Cobert.	0,000	-0,001	0,000	0,001	-0,001	1,000

Leitura da matriz — a independência ficou ainda mais evidente com N maior. O maior $|r|$ de toda a matriz é **0,002** (complexidade ciclomática $\times$ LOC) — ruído amostral de uma amostra de 1.000.000 de registros (erro-padrão de correlação nula $\approx 1/\sqrt{(N-2)} \approx 1/\sqrt{999998} \approx \mathbf{0,001}$ nesta escala), não dependência estrutural. Todos os 15 pares de features têm $|r| \leq 0,002$ — uma ordem de grandeza abaixo do $|r|$ máximo da versão com $N = 150$ (0,087, com erro-padrão esperado $\approx 1/\sqrt{148} \approx 0,082$). Esse é exatamente o efeito estatístico esperado do aumento de amostra: como as 6

features são geradas verdadeiramente independentes (§1.2), aumentar N não muda a correlação populacional (que é 0) — só reduz a variância da estimativa amostral, então o $|r|$ observado converge para 0. Isso reforça, sem mudar, a alegação já feita na versão anterior deste relatório: confirma que o gerador implementa de fato a decisão "Naive Bayes puro" — nenhuma feature herda informação de outra.

Em particular, **complexidade ciclomática × LOC = -0,002**: no dataset da versão original deste projeto (anterior à decisão "Naive Bayes puro") essa correlação era 0,67 (programada de propósito, citando Shepperd, 1988); aqui ela continua deliberadamente nula, e com N = 1.000.000 fica visivelmente mais próxima de 0 do que já estava com N = 150. A discussão sobre o que isso significa — e sobre a correlação real que a literatura documenta para essas duas métricas — é o assunto central da Reflexão Crítica da Etapa 4, não desta etapa.

2.3 Estatísticas descritivas (valor bruto de cada feature)

Feature	Média	Desvio-padrão	Mín.	Mediana	Máx.
Complexidade ciclomática	12,72	11,49	1	9,0	120
LOC	140,39	160,19	8	90,0	1500
Nº de autores	2,55	1,24	1	2,0	9
Churn relativo (%)	13,83	11,55	0,4	10,5	85,0
Nº de imports	7,76	5,24	0	6,0	40
Cobertura de testes (%)	62,80	18,50	2,0	63,0	99,0

LOC e complexidade continuam com distribuição assimétrica à direita (média > mediana), como esperado para métricas de tamanho de código — isso não mudou; o que mudou é que as duas deixaram de crescer juntas (§2.2).

2.4 Taxa de defeito por categoria, feature a feature

Como o **rótulo** ainda depende de cada uma das 6 features isoladamente (é o que o Naive Bayes modela — só deixou de haver dependência *entre* features), faz sentido, e continua valendo, checar se a taxa de defeito por categoria é monotônica e razoável para cada feature individualmente. Taxa geral = 35,0 %:

Feature	baixo	medio	alto
Complexidade ciclomática	25 %	43 %	59 %
Nº de autores	21 %	32 %	58 %
Churn relativo	25 %	41 %	63 %
Nº de imports	28 %	33 %	42 %
LOC	44 %	30 %	36 %
Cobertura de testes	40 %	34 %	30 %

- **Complexidade, autores, churn e imports**: taxa de defeito cresce de forma **monótona** com a categoria, como esperado dos pesos definidos no gerador (§1.4) — sinal preditivo individual claro e coerente com a literatura de cada feature (Seção 3.2 do relatório da Etapa 1). Com N = 1.000.000 essas taxas ficaram ligeiramente diferentes das da versão com N = 150 (ruído amostral menor), mas a forma monótona — o que importa para a análise — é a mesma.
- **LOC**: relação **não monótona, em U** (baixo 44 %, médio 30 %, alto 36 %) — de propósito, não é um efeito colateral. Reflete o peso programado a partir de Koru et al. (2008): módulos pequenos são proporcionalmente mais propensos a defeito, então **loc = baixo** pesa mais que **loc = medio**. Isso é uma propriedade da relação entre a feature LOC **isolada** e o rótulo — não depende de LOC estar ou não correlacionada com outra feature.
- **Cobertura de testes**: sinal fraco e praticamente sem tendência clara entre as categorias (40 % / 34 % / 30 %) — reproduz de propósito a fraqueza documentada dessa feature na literatura (Inozemtseva & Holmes, 2014; Gren & Antinyan, 2017).

Não há mais uma subseção de "checagem de padrões programados entre features" (como a subpopulação "pequeno e denso" da versão anterior) — ela deixou de existir porque essa dependência entre features foi removida do gerador (§1.2). O que resta, e continua válido, é a checagem acima: cada feature, sozinha, ainda empurra o rótulo na direção esperada pela literatura.

3. Conformidade com os Requisitos da Atividade

Requisito (Etapa 2 do enunciado)	Situação
Pelo menos 100 registros	1.000.000 de registros (<code>N_REGISTROS = 1_000_000</code>)
Padrões intencionais e realistas, não aleatórios	Cada feature segue uma distribuição deliberada (§1.4) e o rótulo depende de todas as 6 de forma coerente com a literatura (§2.4) — o que não é intencional, por decisão de projeto, é a correlação <i>entre</i> as features (§1.2)
Distribuição razoável entre classes, sem desbalanceamento completo	35,0 % / 65,0 % ($\approx 1:2$)
Usar apenas as features discretizadas definidas na Etapa 1	6 features, mesmas faixas <code>baixo/medio/alto</code> do <code>CLAUDE.md</code> ; CSV traz valor bruto e categoria
Dados salvos em tabela incluída no entregável	<code>dados/etapa2_dados_treinamento.csv</code> (1.000.000 de linhas + cabeçalho). Arquivo grande, reproduzível via semente fixa — não versionado no Git (<code>.gitignore</code>); ver nota em §1.3 e <code>CLAUDE.md</code>

Reprodutibilidade. Rodar `python3 dados/gerar_dados.py` regenera exatamente o mesmo CSV e o mesmo relatório de validação (semente fixa `SEED = 42`, ramificada por feature via `rng.spawn()`). Os parâmetros no topo do script (`N_REGISTROS`, `SEED`, `PROPORCAO_DEFEITO` e os blocos do modelo generativo) permitem ajustar a massa sem tocar na lógica. Com $N = 1.000.000$ e a geração totalmente vetorizada em numpy (sem laço `for` linha a linha), o script roda em poucos segundos.

Uso de IA. O script, os parâmetros do modelo generativo e este relatório foram construídos em diálogo com uma IA generativa. A decisão de tornar as features independentes entre si ("Naive Bayes puro") foi tomada e documentada no `CLAUDE.md` antes desta geração; os pesos do rótulo por categoria foram derivados das referências verificadas na Etapa 1 (Shepperd, 1988; Koru et al., 2008; Nagappan & Ball, 2005; Inozemtseva & Holmes, 2014; Gren & Antinyan, 2017) e **conferidos na saída de validação** antes de fixar os valores. Todo o código é comentado e defensável linha a linha.